Harmonic Ambiguity in AI-Assisted Melody Generation

Aidan M. Therrien

School of Music, University of Maryland

MUSC448C: AI and Music

Dr. Jessica Grimmer

May 7, 2026

The dominant critique of AI music generation tools like Suno and Udio is not that they fail technically, it is that they succeed in the most forgettable way possible. Producers and musicians have taken to calling this output "coworker music": competent, inoffensive, and completely unmemorable. These tools learn from an enormous dataset of recordings and reproduce the surface texture of genre without any of the structural decisions that make a given song worth returning to. For this project, I wanted to ask a more specific question: what would it look like to build a generator that actually targets the harmonic and melodic qualities that make certain pop songs impossible to forget?

Rather than training on millions of tracks, this project encodes the compositional logic of a single songwriter, Kurt Cobain, as a set of deliberate constraints. Cobain's practice is a useful subject because it exemplifies a particular economy of songwriting and music production: maximum emotional effect from minimum production help or musical complexity. His published notebooks reveal someone obsessed with melodic hooks while working with an intentionally stripped-down harmonic vocabulary. The cornerstone of that vocabulary is the power chord — a dyad of root and perfect fifth with no third, which means no major or minor quality. This is not just a stylistic choice for angry sounding distorted guitar; it is a structural one with real consequences. A power chord carries no tonal color of its own, so the melody above it bears the entire burden of modal identity. The same E5 → A5 → G5 → C5 progression can feel menacing under an Aeolian melody, open and bright under a Mixolydian one, or tense and disorienting under a Phrygian one. The chords establish rhythm and root motion; the melody determines everything else.

This is the central argument of the project, and it is one that current AI music tools are entirely unequipped to make. Suno does not distinguish between a chord's harmonic function and

a melody's modal identity as separable parameters; instead, everything is generated together, and mode is just an emergent property of statistical correlation in the training data. If you want the same chords to sound more Dorian, there is no lever to pull. This generator exposes mode as a first-class parameter precisely because, in rock and grunge, it is one.

The generator is implemented in Python as a package with a Jupyter notebook interface. For MIDI I/O, I used mido, a low-level library that gives direct control over note timing, velocity, and channel routing — I needed that precision because the project requires specific articulation control, with chord notes gating at 88% of their duration and melody notes at 92%, to keep each voice legible in the mix. A higher-level wrapper would have buried those controls. Pitch vocabulary and probabilistic sampling are handled with numpy; melody generation uses a temperature-scaled softmax over pitch indices, where low temperature produces predictable, contour-following lines and high temperature flattens toward randomness. For audio, FluidSynth renders the MIDI to WAV using the GeneralUser GS soundfont — I kept synthesis as an external binary call rather than pulling it into the Python process to avoid DSP complexity the project did not need. Visualization and the interactive notebook interface use matplotlib and ipywidgets, which let a user adjust parameters and immediately see and hear the result as a piano roll and audio widget.

Melodic structure is built around three phrase types: question phrases ascend stepwise from an anchor pitch toward a tension goal, answer phrases descend back to the anchor, and pedal phrases hold a single pitch with occasional neighbor-tone ornaments. Cycling through question and answer phrases creates a call-and-response arc that shows up throughout Cobain's melodic writing — the ascending figure in About a Girl, the descending resolution of Come as You Are, the static repeated tones of Lithium's verse. The generator also tiles a short motif across

the full section rather than producing fresh material each bar, which encodes one of Cobain's clearest habits: his melodies are built from repetition, not development. Repetition is not laziness here, it's fundamental to writing a catchy song. A syncopation parameter shifts note onsets slightly ahead of the beat to simulate the pushed vocal phrasing that gives grunge singing its characteristic forward momentum.

The output is intentionally incomplete. The generator produces a MIDI file with chord and melody tracks, rendered to audio for demo purposes, and nothing more. There are no lyrics, no arrangement decisions, no production choices, instead those are left to a human musician. This is deliberate. The decisions that most determine whether a song is memorable are exactly the ones no current AI tool makes well, and routing them through an algorithm would likely produce the same forgettable result we started with. The last thing we need is more AI generated music where the singer always sounds like Justin Timberlake.

The limitations of this project are worth being honest about. Cobain did not only write four-chord loops — his riffs carry harmonic information through intervallic motion, and a generator that only models power-chord progressions misses that dimension of his writing entirely. More fundamentally, expressive melodic phrasing is still a genuinely hard problem. The phrase grammar produces structurally sound contours, but a real singer introduces rhythmic nuance, unexpected leaps, and micro-variations that collectively create the sense that a melody is being felt rather than calculated. Temperature and syncopation add randomness, but randomness is not the same as expressiveness. More sophisticated probability distributions over rhythm and dynamics, informed by closer analysis of Cobain's recorded performances, would improve this substantially. Still, the core point stands: building around explicit structural logic, even

simplified, produces output that is more musically legible, and more genuinely useful to a collaborating musician, than statistical interpolation at scale ever will be.